RelockerDFS: A Secure Distributed File Store Solution

Ryan Lanciloti

***Abstract*—A problem exists with modern cyber-security in distributed file systems inherent to how we store data. If one system which stores user data is compromised, then that data is also compromised. Some systems will encrypt the user data at rest, but root access to these systems grants access to private keys and passwords which could decrypt this user data. I am proposing RelockerDFS which is a secure, distributed file store which is aimed towards ensuring data privacy is maintained when data is stored off of the host machine. Throughout this paper, I will explain what RelockerDFS is, how it works, and why it is secure.**

I. Introduction

A Problem exists with modern distributed file systems which is that data availability and speed of recoverability is prioritized over data security. In most applications, this would (and should) be seen as acceptable. Most modern systems want to guarantee data reliability through data replication across multiple platforms. However, there are also scenarios where data security should hold the highest priority, and high data volatility (the measure of how easily data loss can occur) is desired. In the small likelihood that there is a data breach, the end user can destroy the data before the bad actor can gain access to any additional data or even make use of the data they did steal.

To achieve this level of security, I propose RDFS, or RelockerDFS, which is secure, distributed file store capable of maintaining secure storage across multiple servers. A "relocker" is the mechanism in a safe which ensures that if a brute force attack against the safe is attempted, the relocker mechanism will trigger and irreversibly lock down forever. This prevents entrance to the safe even through it's intended means.

RDFS utilizes multiple layers of security including AES, RSA, and hashing algorithms. One of the main reasons why data security can be maintained when the data is off-host is due to the fact that the servers storing data are ignorant to the data's significance. They have no way of decrypting the data that they are holding which means the original host needs to be compromised in order for any leaked data to be considered meaningful. The reason why compromised data does not necessarily mean the attacker can gain access to the original data is because files which are uploaded to RDFS are split into multiple pieces and stored separately, being only recombined into one file upon retrieval. This means if one server holding a piece of a file is compromised, the user can delete one or more of the other pieces and the original file can never be recreated. This brings up the next point which is that data isn't inherently replicated across multiple servers upon upload. This ensure that data destruction can be easily achieved if desired.

Due to the fact that RDFS sits upon an existing file system, it is more of a file storage solution as opposed to a file system which can grow dynamically as files are added to it. The file store is accomplished by using "vaults", or files which store smaller pieces of data called blocks. These vaults themselves represent a virtual, fixed-size file system that lives on top of the host operating system. This provides the user flexibility as to where they would like their vaults to be stored at.

II. Related Works

There are some related works that already exist is the distributed file system space. The idea of splitting files into smaller pieces and storing those pieces on different servers is not a novel approach as demonstrated by the authors of "JigDFS: A Secure Distributed File System" [1]. JigDFS uses an "Information Dispersal Algorithm" to split files into n segments, where k of those segments are needed to reconstruct the file. RDFS does not have intrinsic redundancy as this makes data destruction in the event of an intrusion much harder. One thing that JigDFS does do that RDFS doesn't which could be seen as an improvement is that it recursively divides up files and distributes the pieces across different nodes. So RDFS creates fixed-sized blocks of 256 bytes, whereas JigDFS creates n blocks that are equal to the file size divided by n. Then, JigDFS sends the blocks to servers which will divide those blocks up further into n more blocks until either a desired depth is achieved, or the blocks are a minimum size. This is an interesting approach, but it would require additional block tracking and would make it harder to decouple blocks from individual servers [1].

Distributed file systems exist today, one of which is the "Hadoop Platform", a large-scale distributed file system and data analysis platform. Hadoop wasn't originally designed with security in mind, however efforts have been made to add transparent encryption to the system using a traditional Key Management Service (KMS). This is known as HDFS. This solution also uses a "convert file into block storage" approach for storing files across different "DataNodes". One of the key differences between HDFS and RDFS is the centralized KMS that generates all the encryption and decryption keys. This centralized authority, if attacked, could bring the entire network to a standstill for all users, whereas in RDFS there is no centralized authority holding a master key. If 1 of 20 servers is attacked, the files that have blocks living on said servers may be lost, but otherwise the network is still functional. Also with

RDFS, as discussed in the *Future Work/Improvements* section, if redundant blocks are created during the file upload process, data loss may not happen due to one server going down [3][4].

III. How RDFS Works

RDFS is best understood through the explanation of a series of mechanisms, where each mechanism is an action that can be taken by the end-user. Across each mechanism, there are repeating concepts which are important in understanding the system as a whole. There is the idea of a **User**, one or multiple **Server(s)**, one or multiple **Vault(s)**, and of course, individual **Files**. Broadly speaking, the relationship between these concepts are as follows: A **User** encrypts, and then splits, **Files** into file fragments which are upload to **Servers**, which then store the file fragments in **Vaults** local to each **Server**. The mechanisms which exist in RDFS are as follows:

- A **User** is created.
- A **Server** is created.
- A **Vault** is created.
- A **File** is uploaded to the network.
- A **File** is retrieved from the network.
- A **File** is removed from the network.

IV. Mechanism I: A User Is Created

In order for the network to do anything of use, it needs users to interact with it. At a high level, a **User** is nothing more than JSON file which follows a specific schema that when interpreted correctly, can be used to interact with the RDFS network. I will refer to this JSON file as the user profile. This user profile will contain no personally identifiable information, and will never be directly interpreted by the end-user during normal operation.

A. Step 1: User Provides A Password

Each user profile is encrypted at rest using AES, where the password is chosen by the end-user. Not storing the password on the file system prevents the decryption key from being recovered via a root or admin account, and therefore the password either needs to be brute forced or coerced out of the end-user. It should be noted that interfacing with a yubikey could provide an optional form of 2-factor authentication which would work by double-encrypting the user profile at rest, first with the user password, and then with the yubikey provided password.

The password chosen by the end-user should be at least 10 characters, containing a mixture of upper-case and lower-case letters, numbers, and symbols. This should ensure a high level of security against brute-force attacks against the user profile. Once a sufficient password is provided, this password should be hashed using SHA-256, and this hash is what is used as the AES key for encrypting the user profile. We will call this key $K_{UserProfile}$.

B. Step 2: User Profile is Generated

At this point, a JSON file can be constructed in RAM, encrypted using $K_{UserProfile}$, and then written to disk. The schema for this file should be as follows:

```
{
    "USER_ID": "USER_UUID",
    "VAULTS": [
        { "VAULT_UUID_1": "RSA_PVT_KEY_1" },
        ...,
        { "VAULT_UUID_N": "RSA_PVT_KEY_N" }
    ],
    "FILE_MAP": {
        "FILE_NAME_1": "FILE_UUID_1",
        ...,
        "FILE_NAME_N": "FILE_UUID_N",
    },
    "FILES": {
        "FILE_UUID_1": {
            "FILE_ENCRYPT_NONCE":
                ↪ "RAND_NUM_128_BITS",
            "PRNG_NONCE": "RAND_NUM_128_BITS",
            "BLOCKS": [
                { "BLOCK_1_UUID":
                    ↪ "VAULT_UUID_X" },
                ...,
                { "BLOCK_N_UUID":
                    ↪ "VAULT_UUID_Y" }
            ]
        },
        ...,
        "FILE_UUID_N": {
            "FILE_ENCRYPT_NONCE":
                ↪ "RAND_NUM_128_BITS",
            "PRNG_NONCE": "RAND_NUM_128_BITS",
            "INITIAL_BLOCK_STORE_ORDER": [1, 2
                ↪ , 4, 3],
            "BLOCKS": [
                { "BLOCK_1_UUID":
                    ↪ "VAULT_UUID_X" },
                ...,
                { "BLOCK_N_UUID":
                    ↪ "VAULT_UUID_Y" }
            ]
        }
    }
}
```

For a new file, however, this profile should only contain the **USER_ID** field with a generated UUID, the **VAULTS** field as an empty list, the **FILE_MAP** field as an empty dictionary, and the **FILES** field also as an empty dictionary.

The **USER_ID** will be used in later mechanisms to associate a profile's vaults to the profile itself when the vaults are located on different servers. Since this is a UUID, it holds no personally identifiable information, yet it will uniquely identify the end-user across all servers and transactions. Since the contents of this file will never be directly interpreted by the end-user, they cannot knowingly give away their profile's UUID. This provides a certain level of deniability to the end-user. When the user asks a server to create a vault, an entry for the given vault will be added to the **VAULTS** field; I will expand on this in the *A Vault is Created* section. The **FILE_MAP** and **FILES** fields will hold data for the files that

the end-user decides to uploads; I will expand on this in the *A File is Uploaded to The Network* section.

V. Mechanism II: A Server is Created

A **Server** is like a Shepard, and the **Vaults** are the sheep; the server's main job is to create vaults and store encrypted file fragments in said vaults. One important note about servers is that they are completely ignorant to the data they are storing, and for who they are storing it. This is important as it provides any one server complete deniability of what it's holding. Much like how a **User** is nothing more than a user profile, a **Server** is nothing more than a server profile. In fact, the same profile encryption methods used for the user profile are also used for the server profile. Encrypting the server profile at rest prevents malicious actors from gaining access to this profile even if they were able to obtain root/admin privileges on the server itself. However, the main difference between the user profile and the server profile is that the server profile contains one singular entry: **BLOCK_NAME_UUID**.

```
{
    "BLOCK_NAME_UUID": "UUID"
}
```

BLOCK_NAME_UUID will be used during vault creation to protect the identity of the vaults at rest. Once this profile has been created, the actual server service can run on the remote server at a predefined port where it is ready and waiting to accept requests from clients on the network. It is the amalgamation of servers across the internet that makes up the RDFS network. There is no inherent broadcast mechanism, nor singular master server which knows the location of all leaf servers which will tell clients on the network where different servers live. Opting for this approach allows a server to remain hidden unless a user knows where to look, reducing the chance that it will be attacked.

VI. Mechanism III: A Vault is Created

Once a **User** has been created, as well as at least one **Server**, the end-user can begin creating **Vaults**. On the host machine (aka the end-user's machine), that there is file which holds a newline separated list of all server hostnames, as well as their port (e.g *localhost:9080*). A **Vault** is essentially a fixed-sized, virtual file system which will store individual file fragments. The capacity of each vault is 2^{16} entries and each entry is a 256-byte block. This provides a vault capacity of $\approx$ 16.78 MB of storage.

A. Vault Schema

It should be noted that a vault is essentially a binary file, meaning that it should be referenced like memory, not like an ASCII encoded file. The schema for each vault is as follows:

- 0x000_0000 - 0x000_00FF: ID_Hash
- 0x000_0100 - 0x000_04FF: Public Key
- 0x000_0500 - 0x011_04FF: Index Map
- 0x011_0500 - 0x111_04FF: Data

The first 256 (0x100) bytes of every vault is its ID_Hash, which is derived by hashing the end-user's *USER_UUID* concatenated with the generated *VAULT_UUID* using SHA-256 as the hashing algorithm. The next 1024 (0x400) bytes is an RSA public key derived from a sister RSA private key stored within the end-user's user profile. The end user will generate a new RSA private key for each vault they create. The use for this key will be explained in the *A File is Uploaded to The Network* section. The next 1,114,112 (0x11_0000) bytes is an entry map which will contain 2^{16} 17 byte entries. The first 16 bytes of each entry will contain the *BLOCK_UUID*, and the final byte will represent the length of the block - 1. The reason why it is block length - 1 is because each block can be 256 bytes long and the maximum size of one byte is 255. Therefore, we assume that 0x00 is actually a block of length 1 and 0xFF is a block of length 256. Finally, the next 16,777,216 (0x100_0000) bytes is the data section. Using this schema, we can write and access data using the following operations:

- DATA = *(0x011_0500 + (INDEX * 0x100))
- BlockID = *(0x000_0500 + (INDEX * 0x11))[0:15]
- BlockLength = *(0x000_0500 + (INDEX * 0x11))[16]

Where **INDEX** is the index into the entry map that should be accessed. The **[0:15]** denotes grabbing and using the first 16 bytes of the dereferenced value, and the **[16]** denotes grabbing the 17th and final byte of the dereferenced value.

B. Vault Creation

So the end-user has decided that the user profile they're using doesn't have enough storage available to them. Well, assuming that they know of at least one server, they can request a new vault to be created. The process for this is as follows:

1) Step 1: Generate The Vault Creation Request: The client application generates a *VAULT_UUID* as well as an RSA public and private key pair. The client will then update the user profile section *VAULTS* to now include a new entry which is a key-value pair, where the key is the *VAULT_UUID* and the value is the *RSA_PVT_KEY*. Next, the client will send a request off to one of the servers (it could be randomly chosen or purposefully picked if so desired) to create a new vault with the given *VAULT_UUID* while also providing an RSA public key derived from the *RSA_PVT_KEY*.

2) Step 2: Creating The Vault in RAM: Once the server receives the vault creation request, it will check it's remaining space. If it has enough, it will start the creation process. The first step is to generate the hash H_{ID_Hash} which is a hash of the *User_UUID* concatenated with the *Vault_UUID* using SHA-256 as the hashing algorithm. The next step is for the server to generate the name of the file that represents the vault on the server's file system. To do this, it takes the *VAULT_UUID* and XORs it with it's own *BLOCK_NAME_UUID*. This is so a bad actor with access to the server cannot determine which vault belongs to which file without knowing the server's *BLOCK_NAME_UUID*. Once a filename has been derived, a file can be created. Inside of this file, at the head, the server will write H_{ID_Hash} which will act as a unique identifier for the vault. Next, the server will write the provided public key into the space immediately following H_{ID_Hash}.

3) Step 3: Encrypting The Vault At Rest: Finally the server generates an AES key using H_{ID_Hash} as the key and uses this to encrypt the vault at rest. The reason for using H_{ID_Hash} as the AES key is so that when the server receives a request from a given client for a given vault, the server can easily derive the H_{ID_Hash} = Hash(REQ_CLIENT_UUID + REQ_VAULT_UUID). It can generate the AES key and if it decrypts the vault and sees H_{ID_Hash} at the head of the file, it knows the requested client owns the vault. However, if it decrypts the vault using the derived key and it doesn't match the first 256 bytes of the file, then the server knows that the client requesting the vault isn't the vault's owner and the server cannot read the vault's data even if it wanted.

A vault created by a given server is only tied to said server though its filename. That means if someone, end-user or bad actor, knows which file corresponds to which vault, they can copy or move the vault to another server. This property makes migrating a vault from server to server extremely simple if one has access to the *BLOCK_NAME_UUID*.

VII. Mechanism IV: A File Is Uploaded To The Network

Once an end-user has generated a user profile, has associated itself with one or more servers, and has created some vaults, they can finally start uploading files to the RDFS network. Because of its complexity, this section will be broken down into 4 parts: File selection and encryption, file division, block send-off, and block storage. Parts 1-3 are done on the client, and part 4 is done server-side.

A. File Selection and Encryption

The first step is to select a file for upload. If there is not enough capacity for the given file to be uploaded, the upload should be denied and the end-user should be prompted to generate more vaults until they have enough capacity to store the given file. Once a file has been selected, and the capacity is deemed sufficient, then the end-user needs to select a file password. Note: this password doesn't need to be unique, nor should a strength be enforced on it. It is recommended, however, that a decent password which is different to the user profile password is specified.

From here, the client will generate a *FILE_UUID* which will be used to map the original file name to all operations that are done to the file. The client will then add a key-value pair into the client profile's *FILE_MAP* section, where the key is the original file name and the value is the generated *FILE_UUID*.

Next the client will generate two 128-bit nonces (number-used-once), one which will be used for file encryption *NONCE*$_{ENC}$, and another which will be used in seeding a psuedo-random number generator *NONCE*$_{PRNG}$. Using N1 = *NONCE*$_{ENC}$, the client will then create H1, where H1 is equal to the hash of N1 concatenated with the provided file password using the SHA-256 hashing algorithm, or H1 = Hash256(N1 + FILE_PASSWORD). N1 is effectively acting as a salt in this hash and this is the reason why a different password per uploaded file isn't strictly necessary. The final step here is to take this hash, H1, and use it as an AES key to encrypt the file being uploaded.

B. File Division

To start off, the client adds a new entry to the user profile under the *FILES* section. The entry is a key-value pair where the key is the *FILE_UUID* and the value is another dictionary. Inside of this dictionary are 3 key-value pairs, *FILE_ENCRYPT_NONCE*, *PRNG_NONCE*, and *BLOCKS*. The key *FILE_ENCRYPT_NONCE* should have the value of *NONCE*$_{ENC}$, and *PRNG_NONCE* should have the value of *NONCE*$_{PRNG}$. The key *BLOCKS* should only have an empty list for its value.

From here, the client will generate H2, where H2 is equal to the hash of *NONCE*$_{PRNG}$ concatenated with the provided file password using the SHA-256 hashing algorithm, or H2 = Hash256(*NONCE*$_{PRNG}$ + FILE_PASSWORD). At this point, the client will create a new instance of a pseudo-random number generator and seed it using H2.

1) File Division Algorithm - Step 0 - Setup: The pseudo-random number generator created in the last step will be referred to in this section as G_R as shorthand for "Generator - Random". The random value generated from G_R will be referred to a V_i where i is an arbitrary iteration in the lifetime of G_R. So if a value generated from G_R is v_i, then the next value generated from G_R would be v_{i+1}. The encrypted file to be uploaded will be referred to as F_{Enc} as shorthand for "File - Encrypted". It should be noted that F_{Enc} will be treated as a byte-stream. So $F_{Enc}[0]$ would represent the first byte in the file, and $F_{Enc}[i]$ would represent an arbitrary byte in the file. F_{Len} will represent the size of the file in bytes. For this algorithm, the user would select some value N which will represent the number of active "bins" that data will be placed into. Each "bin" will represent a "block" of data that makes up a file; a block is simply a bin which is no longer being written to. As part of the setup step, N empty bins should be created with the maximum capacity of 256 entries (all initialized to zero), and each entry will correspond to 1 byte. Each bin will be referred to as B_j where $0 < j < N$, and the fullness of any given bin is denoted as $B_{j.LEN}$. So if the third bin's current capacity was 100, then this could be denoted as $B_{3.LEN} == 100$.

2) Step 1 - Generating a Random Value: Using the seeded G_R, the client will generate a random value, v_i, between 1 and N.

3) Step 2 - Writing a Byte From The File To a Bin: The client will then read the next byte, $F_{Enc}[i]$, from F_{Enc} and append it to the bin corresponding to the number generated in step 1 (e.i. if $N = 4$ and $v_i = 3$, then $F_{Enc}[i]$ would go to B_3).

Next the client would check the following condition: "Was this the last byte in the file", or $i == F_{Len}$? If yes, then all bins, are sent to the "block send-off" stage. If no, then the client checks the following condition: "Has the current bin reached maximum capacity?", or $B_{v_i.LEN} == 256$. If yes, then B_{v_i} is sent to the "block send-off" stage and B_{v_i} is replaced with an empty bin. Go back to step 1, now with $i = i + 1$.

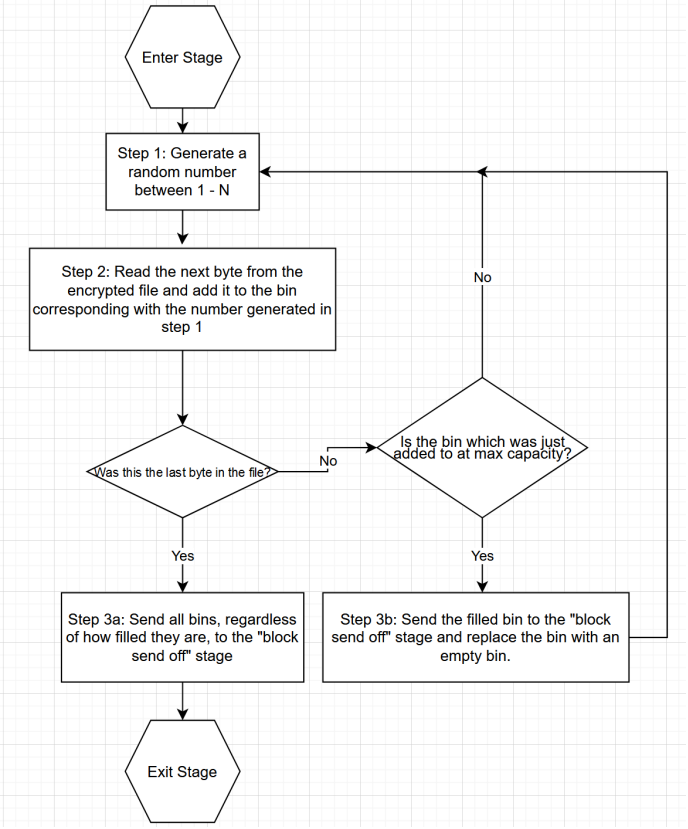


Fig. 1. File Division UML Diagram

C. Block Send-Off

This section is entered either when a block has reached its maximum capacity of 256 bytes, or the entire file has been read through. The first step in this stage is to generate a chained XOR key, K. To generate K, there are 8 intermediate keys, K_i where $1 \leq i \leq 8$, that need to be generated.

1) Generating K_1: Generating K_1 is different depending on which block is being processed. If the block being processed **is the first block for the file**, then the input into this step is the AES key used to encrypt the file. Otherwise, the input into this step is the previously used K. This chaining of the previous output into the input makes the key ”chained”. K_1 is the SHA-256 hash of the input into this step.

2) Generating K_2 - K_8: To generated the intermediate keys 2 through 8, simply take the previous intermediate key and hash it using SHA-256. So $K_i = \text{SHA256}(K_{i-1})$.

3) Building K: To get K, simple concatenate K_1 - K_8, thereby creating a 256-byte key.

4) Encrypting The Block With K: To encrypt the block with K, simply XOR the 256-byte block with the 256-byte key. Since this key is generated with each new block, without the correct AES key used to encrypt the original file, this 256-byte block cannot be read. This key is essentially a one-time pad, and it adds an extra layer of encryption before it is sent off to the server to store it.

5) Sending The Block To a Vault : Now that the block has been full processed on the client-side, the client can prepare to send it off to a vault. The first step is for the client to generate a UUID to identify the block by. The client will then look through its list of available vaults and call out to the servers it knows of and asks who has the vault. If the client receives a response from a server, then it will ask the server if the vault is full. If the vault still has some capacity to it, the client will store a new key-value pair in a list in the user profile under *FILES*[*FILE_UUID*][*BLOCKS*] where the key is the block UUID and the value is the vault UUID. It will then send a message to the server containing the block UUID, a hash of the vault UUID that the block should be stored in, the data to be stored, and the length of that data. The reason the client sends the data length is that not all blocks are guaranteed to be filled to max capacity as they could reach the end of the file beforehand.

If the block being stored is one of the original N blocks, then it is added to the *INITIAL_BLOCK_STORE_ORDER* list. This list exists for determining what order blocks were stored on the network in for file retrieval.

After the initial vault creation, the actual vault UUID is never sent in messages, only a hash of the vault UUID is sent. This is to protect the end-user from having their vault discovered if a message is leaked. The server can get the original vault UUID by XORing its *BLOCK_NAME_UUID* with the files it knows to be vaults, and then it can hash those values to determine which vault the client is referring to. Since vaults are encrypted on the server side using a key which is the hash of the concatenation of the user UUID and the vault UUID, we can prevent nefarious neerdowellers from decrypting vaults by never sending the vault UUID after creation.

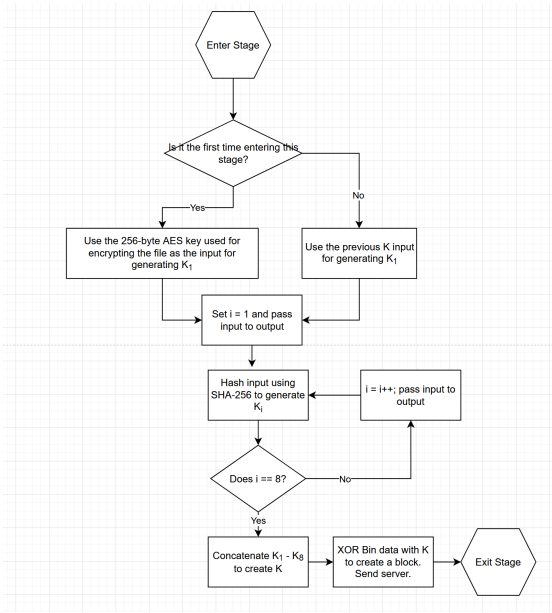


Fig. 2. Block Send-Off UML Diagram

D. Block Storage

Once the block is sent to the server, the server has to store the block in the vault specified. To do this, the server first finds the vault in question and then attempts to decrypt it using the

AES with the key derived from hashing the concatenation of the user UUID with the vault UUID. If the first 256 bytes of the vault match the key, then the server will look for the next entry in the vault's *Index Map* which is all 0s; a block of 0s is interpreted as an empty entry. The server will then put the block UUID provided into the *Index Map* at the found location, as well as the block length. The server will then use the *Public Key* stored within the vault to encrypt the data sent within the message. This way, if the vault is ever decrypted, the corresponding private key would be needed in order to view the actual data stored in the vault. Finally, this encrypted block is written to the correct *Data* entry offset and the vault is closed and re-encrypted once more.

VIII. Mechanism V: A File Is Retrieved From The Network

Once a file has successfully been uploaded to the network, eventually the end-user will want to retrieve the file. As one might expect, retrieving a file is a lot like storing a file but in reverse. There is an important first step before any file can be retrieved: *File Discovery*.

A. File Discovery

The user profile holds a list of every file uploaded by the end-user, however some of the servers which are holding some of the vaults may experience downtime. When a vault is inaccessible, any file which has a part contained in said vault cannot be retrieved by the end-user. So the *File Discovery* process is going through each file in the user-profile, gathering a list of all vaults which hold a piece of a given file, and sending out a request to all known servers to see if one of them has one or more of the required vaults. If the client receives a response for all vaults for any particular file, then it reports back to the end-user that this particular file is retrievable.

B. File Retrieval

Once the client has a list of all files which can be retrieved from the network, the end-user can now select a file from the list. During the selection process, the user will also need to provide a password; this is the same password that will be used in file decryption and seeding the pseudo-random number generator. At this point, the client wants to retrieve the first N-blocks, where N is the number of bins that were open at any given point while the file was being encrypted, from the network. Since the client already knows which vaults are visible on the network, and it also knows which servers replied to the initial query responses, this won't need to be a multi-cast request to the entire network.

One thing to note is the fact that N **needs to be the same in retrieval as it was in upload**. This should be fairly obvious when pondered further, but let's assume a file was uploaded with $N = 4$. If during retrieval $N = 5$, then pseudo-random number generated will be selecting numbers between 1 and 5, not 1 and 4, which means data reconstruction will fail.

Once the client has the first N blocks, it will decrypt each one using the private key which goes with each of the vaults the blocks were stored in. At this point the blocks will be in their pre-send-off state. The client needs to derive the AES key used to encrypt the original file so that it may generate the chained XOR key. The next step is for the client to XOR each block with their correct chained key, and the nice part about how the blocks were stored is that the store order is also the order in which the chained keys need to be derived. At this point, if the client is dealing with the initial N blocks, then the blocks need to be reordered according to the *INITIAL_BLOCK_STORE_ORDER* list. Once reordered, the client can use the seeded PRNG to reconstruct the file in the same way it divided it, but instead of pull each byte from the original file and putting it into a block, it is pulling a byte from the corresponding block and adding to the file. Once all blocks have been read, the original file should now be in its encrypted (but whole) state. The final step is to used the file encryption AES key to decrypt the file into it's original, pre-uploaded state.

IX. Mechanism VI: A File Is Removed From The Network

Once a client has uploaded a file to the network, there may come a time where the user will want to delete the given file. There are many reasons to do this, the most general of which is simply that they want to clear up space. Another possibility is that they want to update a file on the network. Each block is treated as immutable. Once a block has been created and stored in a vault, the only valid operation which can be done on it is a *read*. Therefore, if the end-user wants to upload a new version of a file, they would need to remove the file from the network and then go through the process of re-uploading it.

The steps for removing a file from the network are fairly simple relative to the *File Upload* and *File Retrieval* steps. However, with the way this system is designed, there isn't a great way to guarantee that a file is completely cleaned up. The main cause for this is that this is a distributed system and server/system availability is never guaranteed. Regardless, there are two approaches that could be taken for deleting a file, each with their own pros and cons.

A. Removing Only Files Which Are Retrievable

The first approach, and probably the most straight forward yet most problematic, is to only allow the deletion of files which are retrievable. For a file to be retrievable, the client must have visibility of each server and each vault which holds the blocks that make up the file. The client would simply make a delete request for each block, where it would send the hashed block UUID, the hashed vault UUID, and it's client UUID. Upon confirming that the client requesting the deletion is the owner of the requested vault, it would zero out the correct *DATA* section and *Index Map* entry. So long as the end-user wants to delete a file which is retrievable, this method works perfectly, but what happens when they need to delete a file which is not completely retrievable?

Lets say user A has uploaded a file to the network called *HW1.pdf*. An hour later, they realize they made a mistake and

need to re-upload a new version of the file, however one of the servers they uploaded to is down for maintenance. At this point, the file is not retrievable so it cannot be deleted, so either the end user has to wait until the server comes back online, or they need to upload the file under a different name. Taken one step further, the server that was taken down for maintenance actually had a hard-drive failure which resulted in all data uploaded to it within the past 6 hours getting lost. At this point *HW1.pdf* will never be retrievable since part of the file is lost forever. However, the entry still exists in the user profile and every other block still exists in the other vaults which are accessible. This is obviously a messy case, but it leads into the second approach to deleting files from the network.

B. Removing Files Which Are Partially Retrievable

The second approach is to allow the deletion of files which are partially retrievable from the network. This means there exists at least one block which is visible to the client, and thus one block which can be destroyed. The actual method of deletion here is exactly the same as it was in the previous approach except that the client creates a list of blocks which are visible and request those are deleted. So if the scenario created in the previous approach is considered, the only sections which are visible are the ones which weren't lost in the hard-drive failure. This method of deletion would clean up all the blocks consisting of the original file. It should be noted, however, that the client doesn't know about the hard-drive failure, but yet it will still delete the file entry in the *FILES* section of the user profile without attempting to delete this block.

If we consider an alternative scenario where the server was taken down for routine maintenance and no data loss occurred, yet the file was still deleted, then suddenly there exists an orphaned block. An orphaned block is a vault entry which doesn't map to a particular file and thus cannot be tracked. It is essentially dead space which would require a new orphaned block clean-up mechanism to fix. The client would have to go through each file and determine which block UUIDs map to which vault UUIDs, and store this in a temporary list. It would then, for every vault, need to send a request to it's server which contains a list of the block UUIDs the user profile has in use, and request that any other block UUIDs present in said vault be removed. This approach would effectively solve the previous problem, but it is still far from ideal.

C. The Right Approach

Both approaches have pros and cons, but the second approach is probably the correct approach for a couple of reasons. Firstly, the inability to delete "corrupted" files from the network could result in a lot of wasted spaced. Secondly, with the orphaned block clean-up mechanism, there is a method to reclaim lost space, and while not ideal, it could be integrated into the initial server discovery process which would help save on the amount of time taken to reclaim space. Thirdly, if redundancy is added to the system, and the same block is written to two servers in two locations, then the first approach is no longer valid. This is because the client has no idea how many copies of a block exist across the different servers and thus a successful deletion could still result in orphaned blocks. Therefore, an orphaned block clean-up mechanism is still needed to ensure space isn't lost due to non-visible redundant blocks not getting deleted.

X. Future Work/Improvements

Over the course of writing this paper, I have thought of a handful of future improvements that could be made to RelockerDFS to improve reliability, security, anonymity, and scalability.

A. Reliability

I touched on this topic earlier with the addition of redundant blocks. The idea here is that one block could be stored on the network twice in two locations with the same block UUID. The reason this works is that the blocks are immutable, thus making files immutable. During file retrieval, the client would request a block with a certain block UUID, and whichever server responds first will be the one which the client retrieves the block from. However, while this makes the system more reliable, if the *FILE_ENCRYPTION_NONCE* and *PRNG_NONCE* are not deleted for a given file, and the redundant blocks are not deleted, the file could still be retrieved even after deletion.

B. Security

One issue that exists in the system is the question of what happens when a malicious server is stood up and starts responding to block requests? A malicious server could start responding with bad blocks and thus, generating an output file which would be corrupted. The easiest way to circumvent this would be to hash the data blocks before they are sent off to the server and to store this hash in the *FILES[FILE_UUID][BLOCKS]* list along side the *BLOCK UUID*. During file retrieval, the client hashes the data sent back from the server and verifies that the hashes match. This would allow the user to guarantee block integrity.

C. Anonymity

Currently if a bad actor was sitting on the network, they could listen for all requests and begin to correlate whom is requesting what. This would be a completely passive form of reconnaissance which could be improved if the bad actor in question stood up a malicious server that keeps a log of the IP addresses and user UUIDs and hashed vault UUIDs that are requested. To help invalidate some of this reconnaissance being done, each client and server could be configured to make bad requests to other servers. So a client may randomly send out a request with a random user UUID and hashed vault UUID not expecting any sort of response, but rather to create invalid traffic. And if the user does receive a response from a server, then this could be used to blacklist servers from future requests. Servers could also partake in creating invalid traffic

in an attempt to make it difficult to tell who is a client and who is a server on the network.

Along with this idea, there could be message brokering servers, essentially proxy servers, which would make requests on a user's behalf to hide which user was requesting what information. This could be done in such a way as to guarantee that the proxy cannot read any data from the transaction, but also that the destination server cannot determine the true origin of requests, thereby increasing the overall anonymity of the end-user.

Finally, an older paper "Defeating Encrypted and Deniable Files Systems: TrueCrypt v5.1a and the Case of the Tattling OS and Applications" [2] discusses the idea of a "deniable file system" or a file system which is hidden from view from the host operating system, and the mere presence of a file structure cannot be detected. While the vaults themselves are somewhat deniable file systems for the individual blocks that make up the files, a proper deniable file system on the servers to contain the vault files would improve security by quite a lot. Ideally, there would be a way to hide the vault files from the host operating system, yet allow servers to still read and write from the vault files. This could be done by taking unallocated space on disk and writing data to those addresses directly. So while the host operating system sees no partition, or no file system, within a particular section of disk, raw reads and writes are still performed on it. However, this has it's own problems and engineering difficulties to circumvent before it's considered a "viable" option.

D. Scalability

This solution takes inspiration from DNS, where there is a hierarchical approach to keeping track of servers. As more and more servers are associated with a client, the client having to ask each individual one if it has a given vault will scale at $O(N * M)$ where N is the number of servers and M is the number of vaults. To help reduce this problem, there could be a few management servers which keep track of a subsection of vault storage servers. These management servers could keep some sort of cache of responses, so if the same client requests the same block that the management server knows exists in one of the servers it manages, it can send the request on to that server immediate instead of interrogating every server under it. The reason this is better than a client-side cache is that a server-side cache would persist past the lifetime of the client, which is meant to only exist for a short duration of time.

There are some side effects to this solution though. Firstly, the management servers could enforce redundancy without the user explicitly asking them to. This could be seen as a good thing or a bad thing depending on your position. Secondly, the management server could start sorting vaults on individual servers by UUID to reduce the number of calls it needs to make. While nice on paper, it does detract from the whole "security" portion as where data is stored is now fairly trivial to determine. Thirdly, if this management server goes down, then it cuts off access to a much larger number of vault storage servers. A management server obfuscates the vault storage servers away from the client, so the client would have no idea where to send requests if the management server is taken offline. In general, management servers have a large amount of power in the network and would need to be heavily regulated to ensure that they aren't being used for malicious intent. Also, many of the issues with HDFS discussed in the references [3][4] would also plague the management servers.

XI. Conclusion

Distributed file storage solutions which prioritize absolute data security over data recovery are not common in modern times. Everywhere we turn, it seems like true security is hard to come by, either because companies are secretly selling user data, or because single-points-of-failure exist in the systems we choose (or don't choose) to store our data in on the every-growing cloud. Data leaks occur every day, and it's only a matter of time before a person's sensitive information ends up for sale on the dark web.

RelockerDFS is a solid solution to the problem of absolute security for the individual. This paper has provided a technical overview of different mechanisms that, when combined, create a secure, distributed file storage solution. RelockerDFS is anonymous as no personally identifiable information is used during transactions. It provides plausible deniability to all servers which choose to store the provided data since the data they receive is segmented and encrypted before sent. And it provides high data volatility for situations where a data leak is worse than data loss. There exists a lot of flexibility in this system which can be used to tweak how it works to best suit the needs of the individuals using it.

Does this solve the problem of individual security once and for all? No, not even close. But it does provide a small step forward in reclaiming the ever-shrinking space that is personal privacy.

References

[1] Jiang Bian and Remzi Seker. Jigdfs: A secure distributed file system. In *2009 IEEE Symposium on Computational Intelligence in Cyber Security*, pages 76–82, 2009.

[2] Alexei Czeskis, David J. St. Hilaire, Karl Koscher, Steven D. Gribble, Tadayoshi Kohno, and Bruce Schneier. Defeating encrypted and deniable file systems: Truecrypt v5.1a and the case of the tattling os and applications. In *Proceedings of the 3rd Conference on Hot Topics in Security*, HOTSEC'08, USA, 2008. USENIX Association.

[3] Omar Hussein. Identification of threats and vulnerabilities in public cloud-based apache hadoop distributed file system. In *2019 15th International Computer Engineering Conference (ICENCO)*, pages 44–49, 2019.

[4] Fei Jiang, Zhenggao Pan, Qingzhao Li, Linsheng Huang, and Dongyan Zhang. Research on the application of transparent encryption in distributed file system hdfs. In *2020 19th International Symposium on Distributed Computing and Applications for Business Engineering and Science (DCABES)*, pages 1–4, 2020.